Lecture: Depth-First Search in Directed Graphs

Topics Covered

- DFS in digraphs
- Cycles and acyclic graphs
- DAGs (Directed Acyclic Graphs)
- Topological sort (linear order)

1 DFS on Directed Graphs

We can run DFS on a digraph; we just need to follow edges in the correct direction. We will add a step to the EXPLORE procedure to keep track of a clock and record when we first visit a vertex and when we finish exploring its edges.

1.1 DFS Algorithm with Timestamps

Algorithm 1 DFS(G) for a digraph

```
For all v ∈ V, visited(v) ← false
clock ← 1
For all v ∈ V:
    If not visited(v)
        EXPLORE(G, v)
```

1.2 Example of DFS on a Digraph

Consider the following directed graph:

After running DFS starting from vertex A, we get pre and post timestamps:

Edge types:

- Black edges: Tree edges (solid edges in auxiliary graph)
- Magenta edges: Forward edges

Algorithm 2 Explore(G, u)

visited(u) $\leftarrow$ `true`
pre(u) $\leftarrow$ clock
clock $\leftarrow$ clock $+ 1$
For each directed edge $(u, w) \in E$:
    If not visited(w)
        $\textsc{Explore}(G, w)$
post(u) $\leftarrow$ clock
clock $\leftarrow$ clock $+ 1$

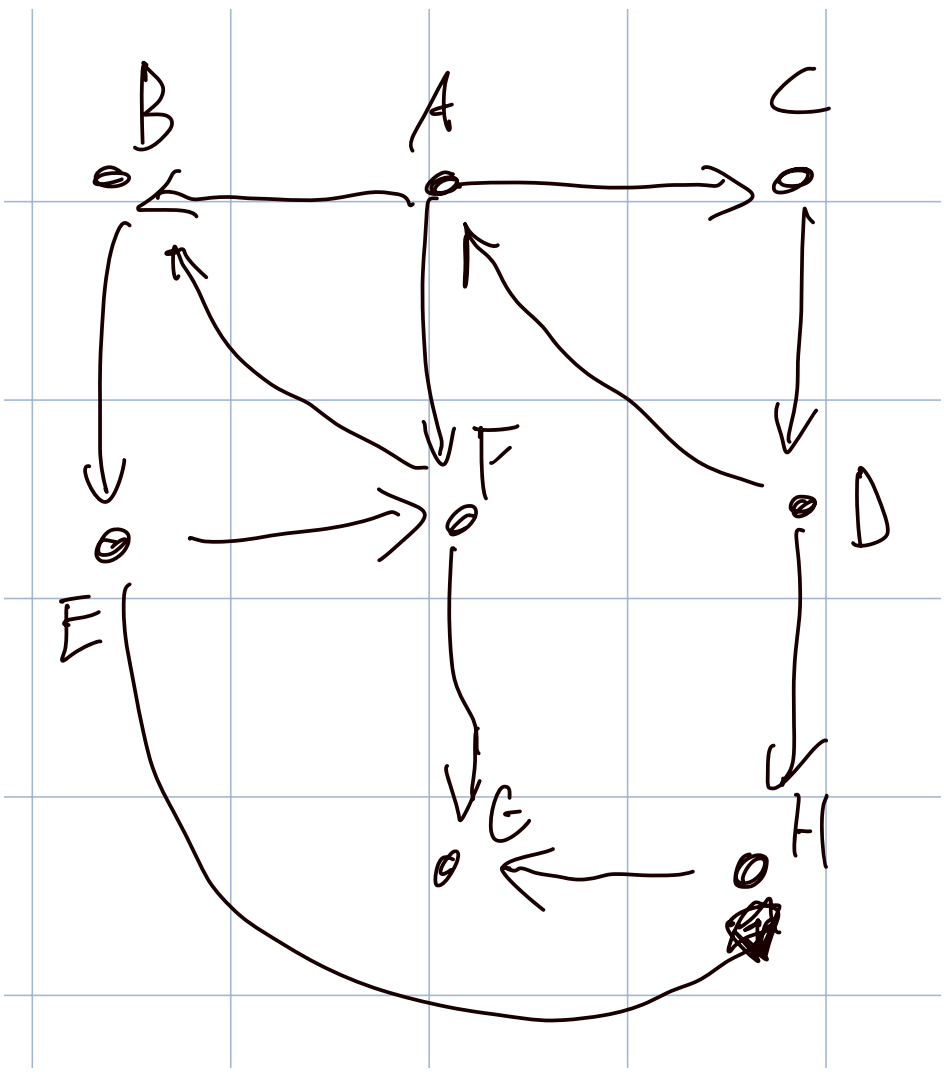


Figure 1: A directed graph

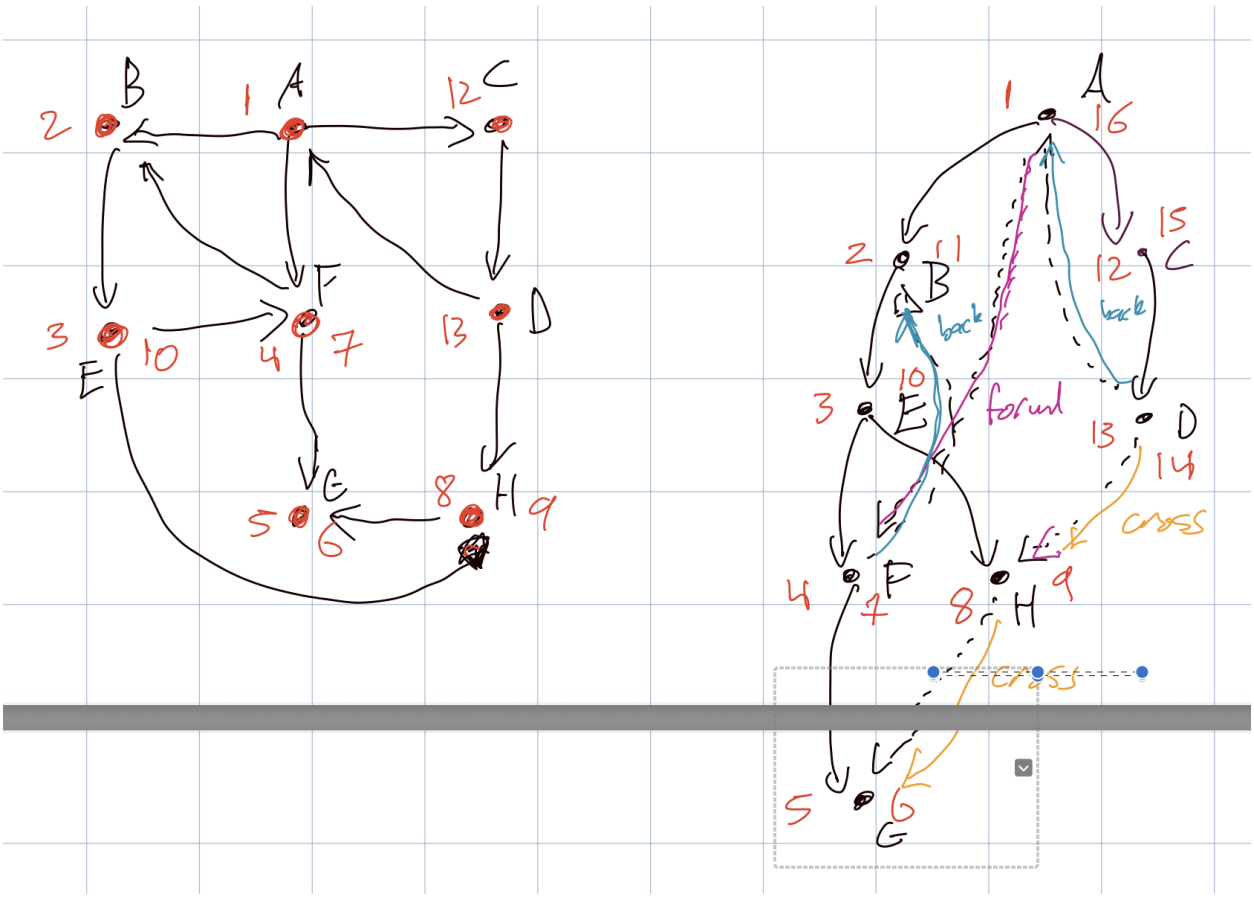
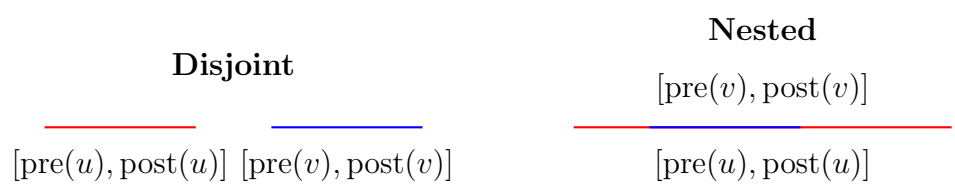


Figure 2: Outcome of alphabetical DFS with auxiliary graph

- Blue edges: Back edges
- Orange edges: Cross edges

1.3 Key Property of Pre/Post Intervals

Theorem 1 (Parenthesis Theorem)**.** *For any pair of vertices u, v, the intervals $[pre(u), post(u)]$ and $[pre(v), post(v)]$ are either disjoint or nested.*



Why is this true? Because when we explore, we place a vertex on a stack. A vertex is removed from the stack only after all its descendants have been explored.

2 Relationships Between Vertices in DFS

Definition 1 (Root)**.** *The **root** is the first vertex explored (A in our example).*

Definition 2 (Ancestor)**.** *u is an **ancestor** of v if there is a directed path of solid (tree) edges from u to v in the auxiliary graph. Equivalently, $[pre(v), post(v)] \subseteq [pre(u), post(u)]$.*

Definition 3 (Descendant)**.** *v is a **descendant** of u if there is a directed path of solid edges from u to v in the auxiliary graph.*

Definition 4 (Parent)**.** *u is a **parent** of v if (u, v) is a solid edge in the auxiliary graph.*

Definition 5 (Child)**.** *v is a **child** of u if (u, v) is a solid edge in the auxiliary graph.*

2.1 Types of Edges

Based on the auxiliary graph (DFS tree), edges can be classified as:

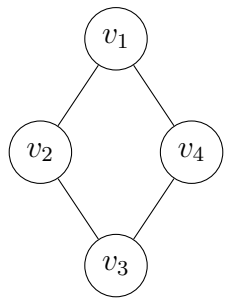
1. **Tree Edge:** Solid edges in the auxiliary graph (edges traversed during DFS, to a not yet visited vertex).
2. **Forward Edge:** From an ancestor to a descendant (not a tree edge). Example: An edge that skips intermediate vertices in the tree.
3. **Back Edge:** From a descendant to an ancestor.
4. **Cross Edge:** Between vertices whose intervals are disjoint. These connect different branches of the DFS tree.

Running time: DFS runs in $O(m + n)$ time (linear time).

3 Cycles, Dicycles, and DAGs

3.1 Definitions

Definition 6 (Circuit in Undirected Graphs)**.** *A **circuit** is a sequence of vertices $v_1, v_2, \ldots, v_k, v_{k+1}$ such that $v_1 = v_{k+1}$ and $\{v_i, v_{i+1}\} \in E$ for all $i \in \{1, 2, \ldots, k\}$.*



Definition 7 (Cycle in Undirected Graphs)**.** *A **cycle** is a circuit in which all vertices except the first and last are distinct.*

Definition 8 (Dicycle in Directed Graphs)**.** *For directed graphs, the definition is exactly the same, but we use directed edges. A directed cycle is called a **dicycle**.*

Definition 9 (Acyclic Graph)**.** *A (directed) graph is **acyclic** if it contains no cycles (dicycles).*

3.2 Characterization of Acyclic Graphs

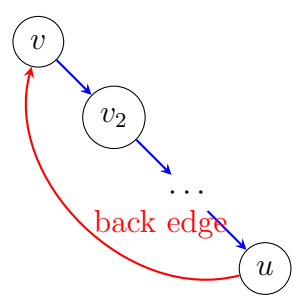
Lemma 1. *A directed graph is acyclic if and only if DFS has no back edges.*

Proof. We prove both directions.

($\Rightarrow$) If DFS has a back edge, then G has a dicycle:

Suppose DFS has a back edge, call it (u, v). Then u is a descendant of v. So there is a directed path of tree edges from v to u, call it $v = v_1, v_2, \ldots, v_k = u$.

We can form the cycle: $v, v_2, \ldots, v_k = u, v$ using the path from v to u and the back edge from u to v.



($\Leftarrow$) If G has a dicycle, then DFS has a back edge:

Suppose G has a dicycle, call it $v_1, v_2, \ldots, v_k, v_{k+1} = v_1$.

There is some vertex, call it v_j, that is the first vertex visited on this cycle. All other vertices in the cycle are descendants of v_j in the DFS tree.

Then the edge (v_{j-1}, v_j) is a back edge, since v_{j-1} is a descendant of v_j (it's visited after v_j and is part of the cycle), and there's an edge from v_{j-1} back to v_j. $\square$

4 Directed Acyclic Graphs (DAGs)

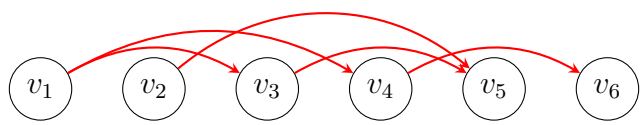
Definition 10 (DAG)**.** *A **Directed Acyclic Graph (DAG)** is a directed graph with no dicycles.*

DAGs are important in applications involving time or causality, where edges represent dependencies or precedence relationships.

4.1 Topological Sort (Linearization)

Definition 11 (Topological Sort)**.** *A **topological sort** (or linearization or linear order) of a directed graph is an ordering of vertices $v_1, v_2, \ldots, v_n$ so that every edge goes from a lower-numbered vertex to a higher-numbered vertex.*

That is, if $(v_i, v_j) \in E$, then $i < j$.



All edges go forward (left to right)

Lemma 2. *All DAGs have a topological sort, and one can be found in linear time $O(m+n)$.*

4.2 Algorithm for Topological Sort

Algorithm 3 Topological Sort

```
Run DFS on the graph
Sort vertices by post-order label, highest to lowest
return the sorted order
```

Exercise: Show that this gives a valid topological order.

Hint: If (u, v) is an edge, what can you say about post(u) and post(v)? Consider the different types of edges (tree, forward, cross). Note that there can be no back edges in a DAG!

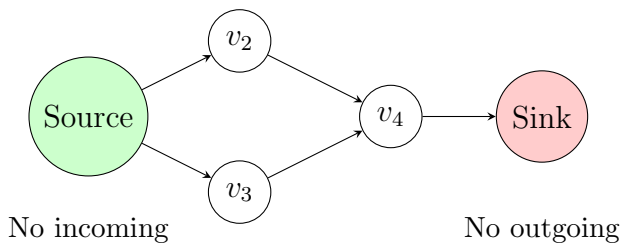
4.3 Special Vertices in DAGs

Definition 12 (Source)**.** *A* ***source*** *is a node with no incoming edges.*

Definition 13 (Sink)**.** *A* ***sink*** *is a node with no outgoing edges.*

Corollary 1. *Every DAG has a sink and a source, and both can be found in linear time.*

Proof. The source is the first vertex in the topological order, and the sink is the last vertex in the topological order. □



5 Summary

- DFS on directed graphs uses pre/post timestamps to classify edges
- Pre/post intervals are either disjoint or nested (Parenthesis Theorem)
- A directed graph is acyclic $\iff$ DFS has no back edges
- Every DAG has a topological sort (found in $O(m+n)$ time)
- Topological sort: order vertices by decreasing post-order number
- Every DAG has at least one source and at least one sink